

CS2103
Software Engineering

Ganesh



- Masters & PhD from National University of Singapore (NUS)
- Several years in Industry/Academia
- Architect, Manager, Technology Evangelist, Educator
- Talks/workshops in USA, Europe, Australia, Asia
- Software Engineering, Cloud Computing, Artificial Intelligence, Game Theory, DevOps
- Holds many industry certifications
- Kathakali Dancer, YouTube Travel Vlogger, Speaker



GANESHNIYER

<http://ganeshniyer.github.io>



Which lines have *coding standard violations*?

```
29  boolean send(int expected) {  
30      String XmlFormat = null;  
31      if (XmlFormat.length() < expected) {  
32          return true;  
33      } else {  
34          return false;  
35      }  
36  }
```

Code quality vs coding standard

Code quality guidelines

Follow a

coding standard
(aka *style guide*)
(aka *conventions*)

subjective vs objective

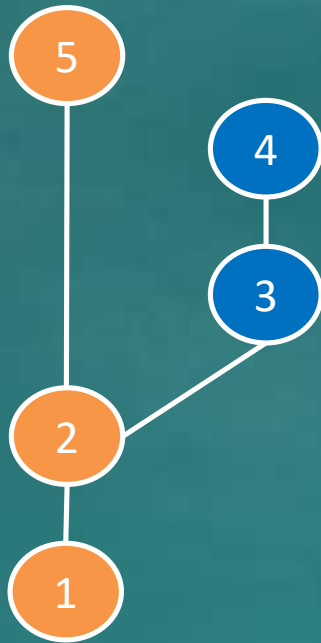
Open the week 4 Canvas quiz

Q₁

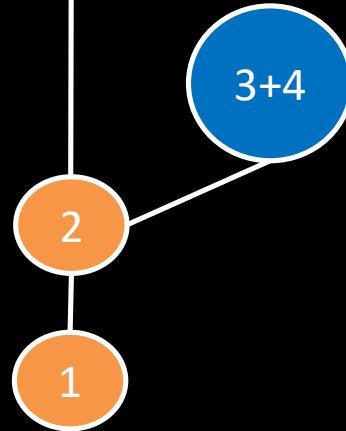
Which one is which?

merge rebase squash

original

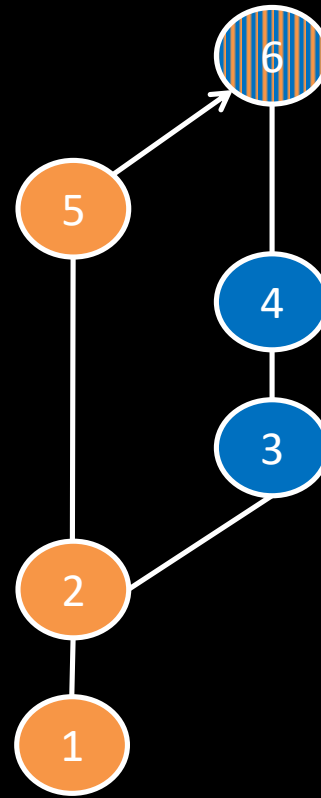


(a)

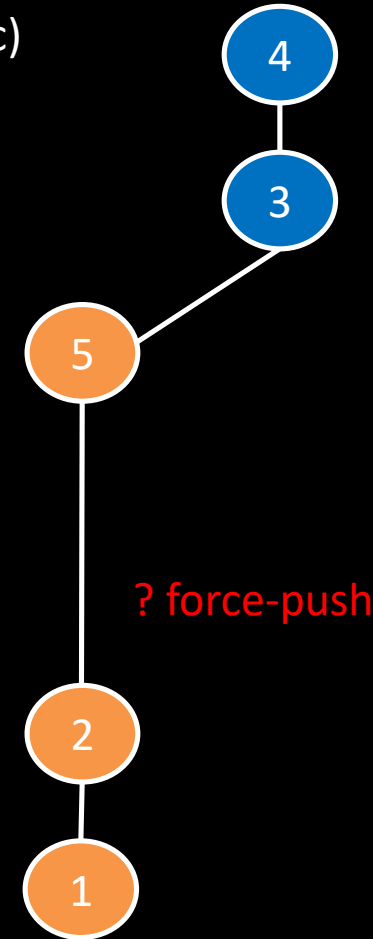


? force-push

(b)



(c)

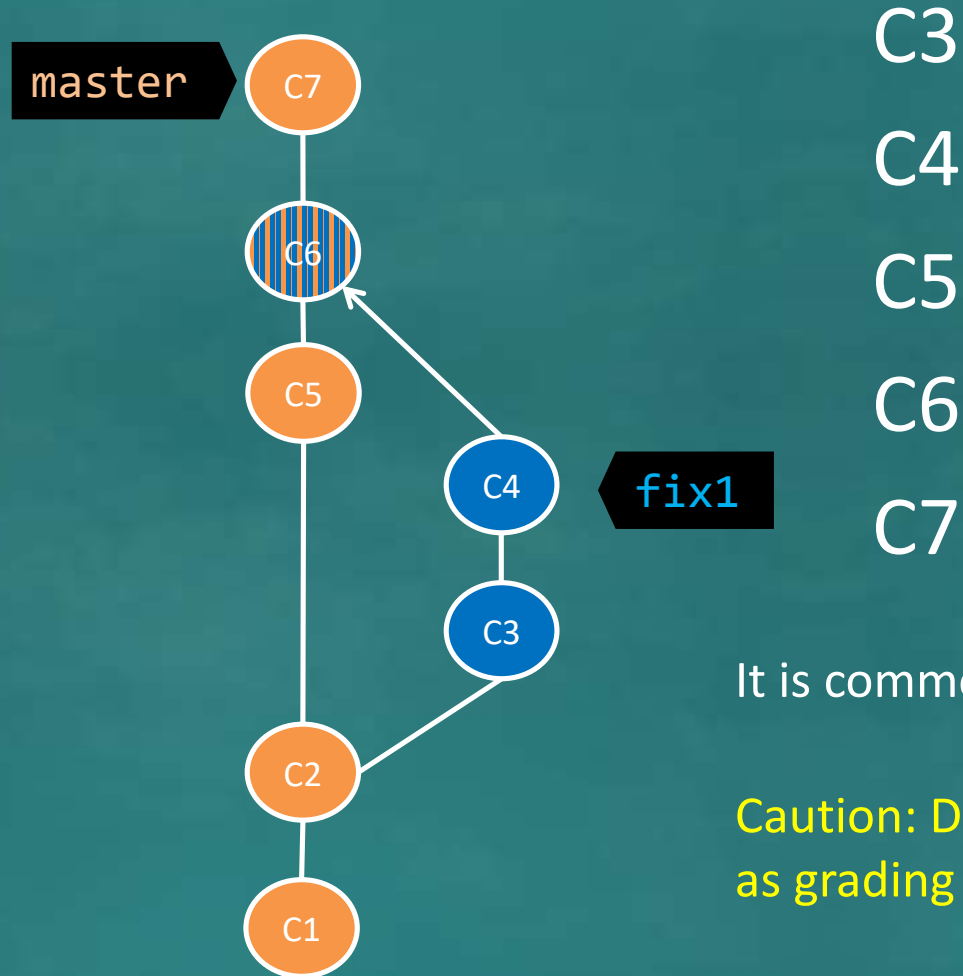


? force-push

The answer is auto-saved.
Don't submit the quiz yet!

Q₂

Which commits will **remain**
if you delete branch `fix1` ?



It is common to delete branches after merging.

Caution: Don't delete branches in the iP and tP,
as grading scripts check for branches

Firefox Source Docs

Firefox

Quick search

ENHANCED BY Google 🔍

OVERVIEW

- A Glossary of Common Terms
- A Quick Guide to Mozilla Applications

GETTING STARTED

- Getting Set Up To Work On The Firefox Codebase

WORKING ON FIREFOX

Working on Firefox

- Firefox Contributors' Quick Reference
- Working with stack of patches Quick Reference
- Pocket Guide: Shipping Firefox
- Editor / IDE integration
- Getting reviews
- Levelling up

How to submit a patch

- Preparation
- Module ownership
- Working on a patch
- Testing
- Submit the patch
- Getting reviews for my patch
- Addressing review comments
- Pushing the change
- Regressions

Working on Firefox / How to submit a patch

How to submit a patch

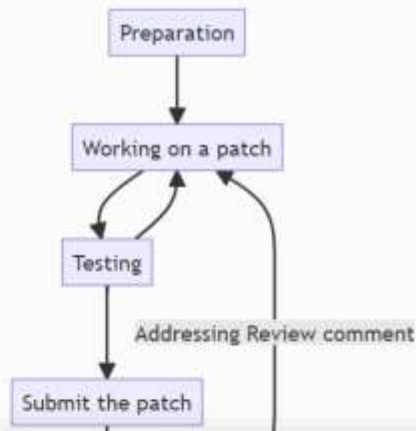
This page is an import from MDN and the contents might be outdated

Submitting a patch, getting it reviewed, and committed to the Firefox

Note

We are also providing a [Firefox Contributors Quick Reference](#) for contributors.

The process of submission is illustrated by the following diagram, and each step is detailed below:



Testing

All changes must be tested. In most cases, an automated test is required for every change to the code.

While we desire to have automated tests for all code, we also have a linter tool which runs static analysis on our JavaScript, for best practices and common mistakes. See [ESLint](#) for more information.

Ensure that your change has not caused regressions, by running the automated test suite locally, or using the [Mozilla try server](#). Module owners, or developers on [Matrix](#) may be willing to submit jobs for those currently without try server privileges.

An alternative to Pull Requests. Git can create a file (called a patch) describing your proposed changes. Git can also 'apply' the patch file to another repo, to add your changes to that repo.

🏠

Firefox Source Docs

Firefox

Quick search

ENHANCED BY Google 🔍

OVERVIEW

A Glossary of Common Terms

A Quick Guide to Mozilla Applications

GETTING STARTED

Getting Set Up To Work On The Firefox Codebase

WORKING ON FIREFOX

Working on Firefox

Firefox Contributors' Quick Reference

Working with stack of patches: Quick Reference

Pocket Guide: Shipping Firefox

Editor / IDE integration

Getting reviews

Levelling up

How to submit a patch

Preparation

Module ownership

Working on a patch

Testing

Submit the patch

Getting reviews for my patch

Addressing review comments

Pushing the change

Regressions

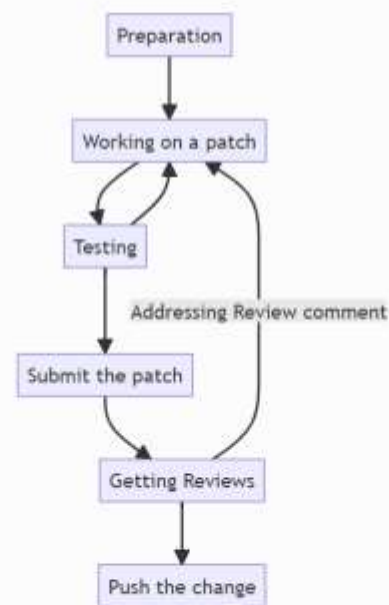
How to submit a patch

This page is an import from MDN and the contents might be outdated

Submitting a patch, getting it reviewed, and committed to the Firefox source tree involves several steps. This article explains how.

Note
We are also providing a [Firefox Contributors Quick Reference](#) for contributors.

The process of submission is illustrated by the following diagram, and each step is detailed below:



Preparation

Every change to the code is tracked by a bug report in bugzilla.mozilla.org. Without a bug, code will not be reviewed, and without review, code will not be accepted. To avoid duplication, [search for an existing bug](#) about your change, and only if none exists, file a new one. Most communication about code changes take place in the associated code review, so be sure the bug describes the exact problem being solved.

```
1  plugins {
2      id 'java'
3      id 'checkstyle'
4      id 'com.github.johnrengelman.shadow' version '7.1.2'
5      id 'application'
6      id 'jacoco'
7  }
8
9  mainClassName = 'seedu.address.Main'
10
11  sourceCompatibility = JavaVersion.VERSION_11
12  targetCompatibility = JavaVersion.VERSION_11
13
14  repositories {
15      mavenCentral()
16      maven { url 'https://oss.sonatype.org/content/repositories/snapshots/' }
17  }
18
19  checkstyle {
20      toolVersion = '10.2'
21  }
22
23  test {
24      useJUnitPlatform()
25      finalizedBy jacocoTestReport
26  }
27
```

Gradle can manage dependencies too!

Lecture briefing structure

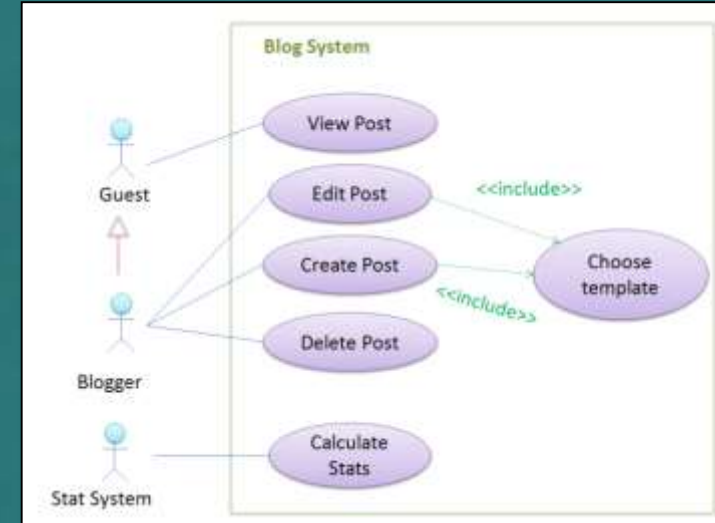
Recap of the last week 

+

Preview of the next week

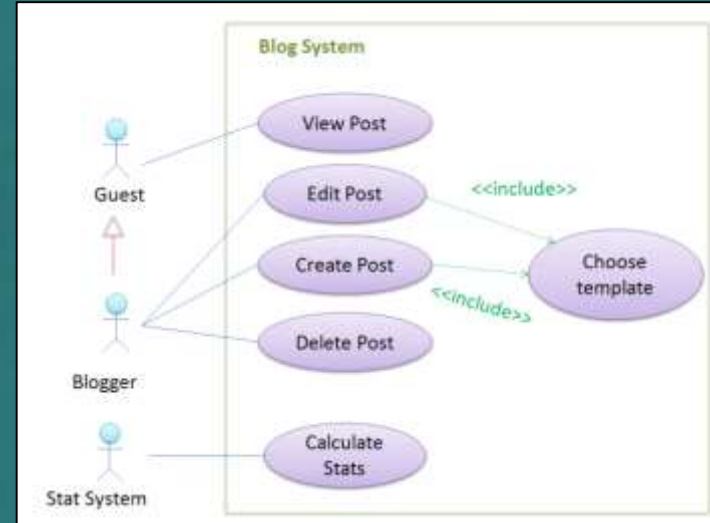
Q₃

What word is common to all three?



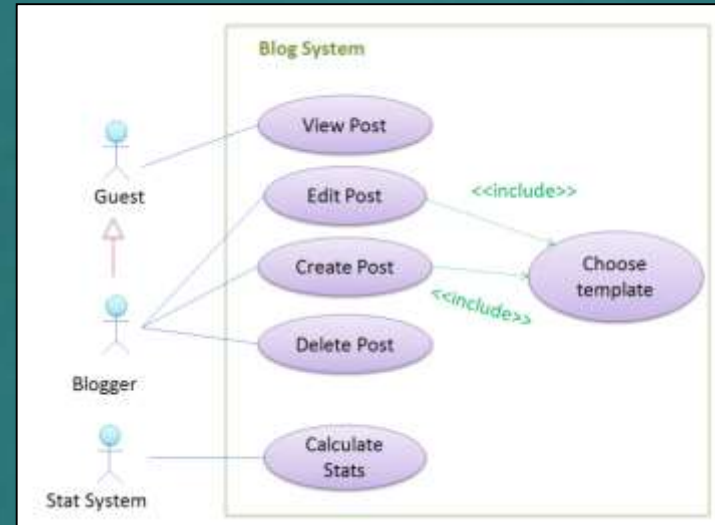
- a) forward
- b) diagram
- c) model
- d) person

What word is common to all three?



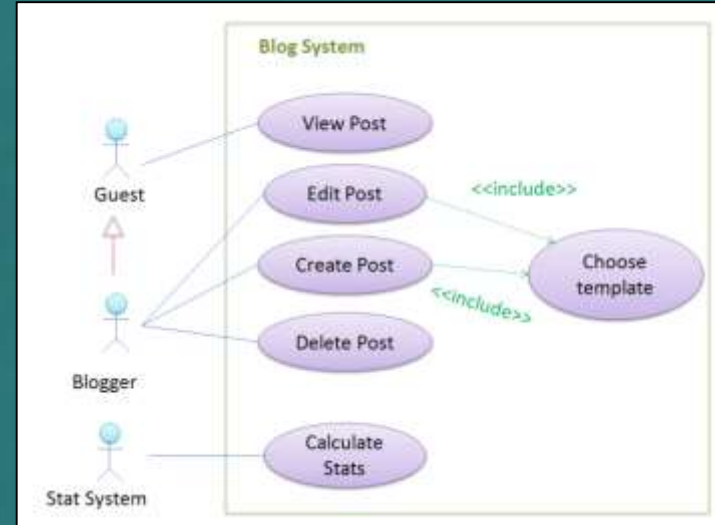
models

[... are representations of something else]



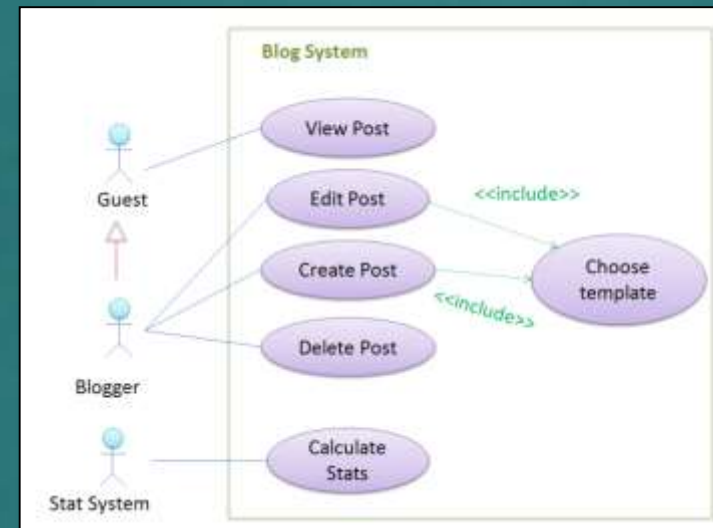
models

... are **not the real thing!**

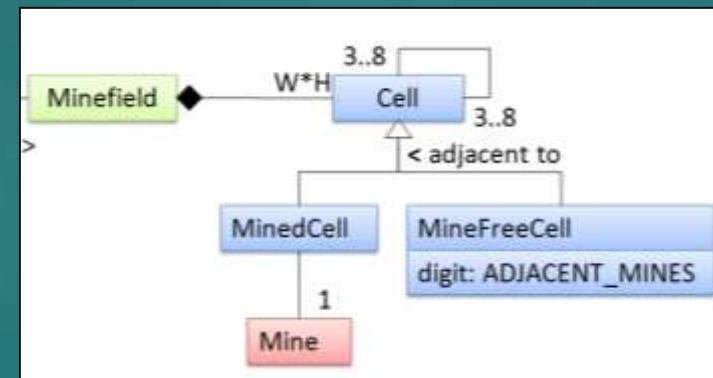
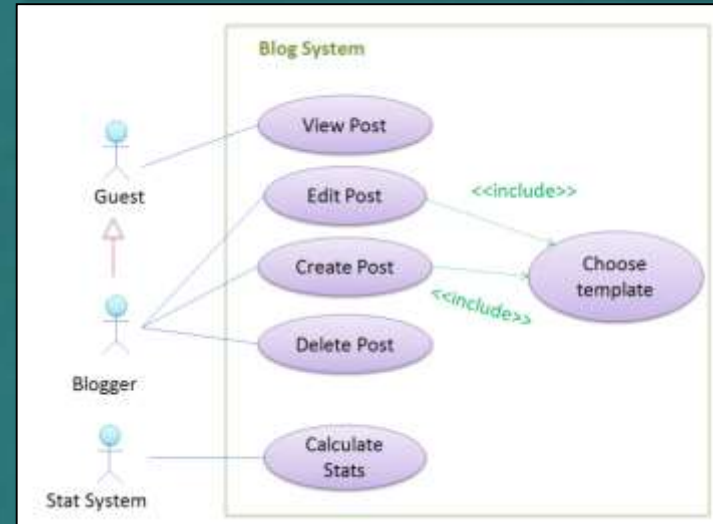
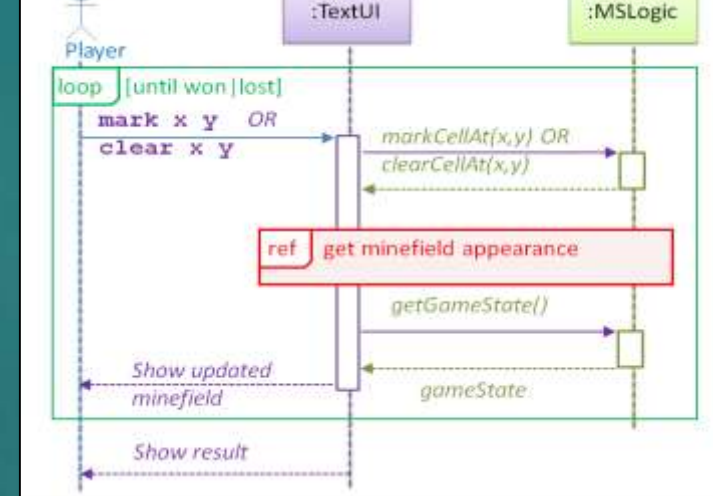


models

... are **simpler than** the real thing!



models



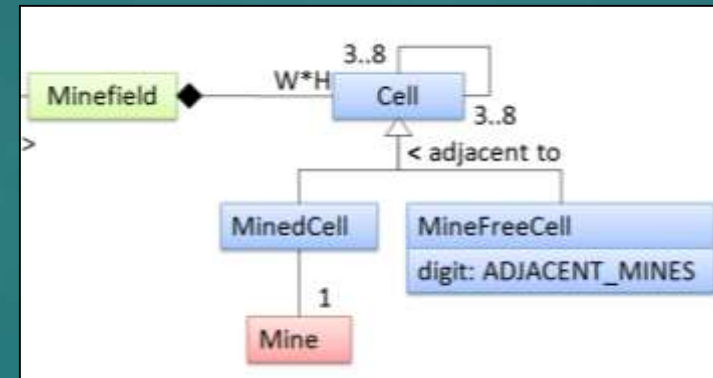
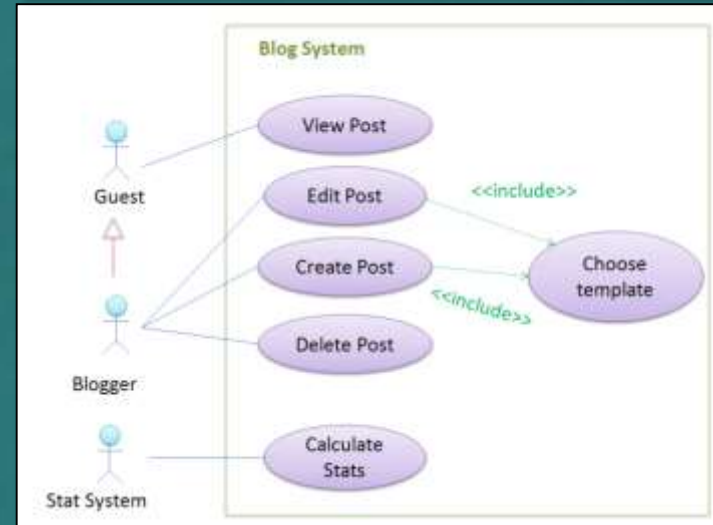
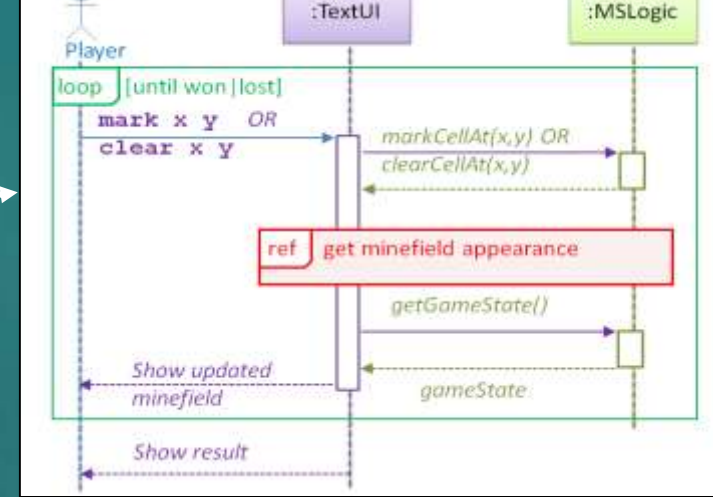
models

... capture different aspects



models

... combat complexity

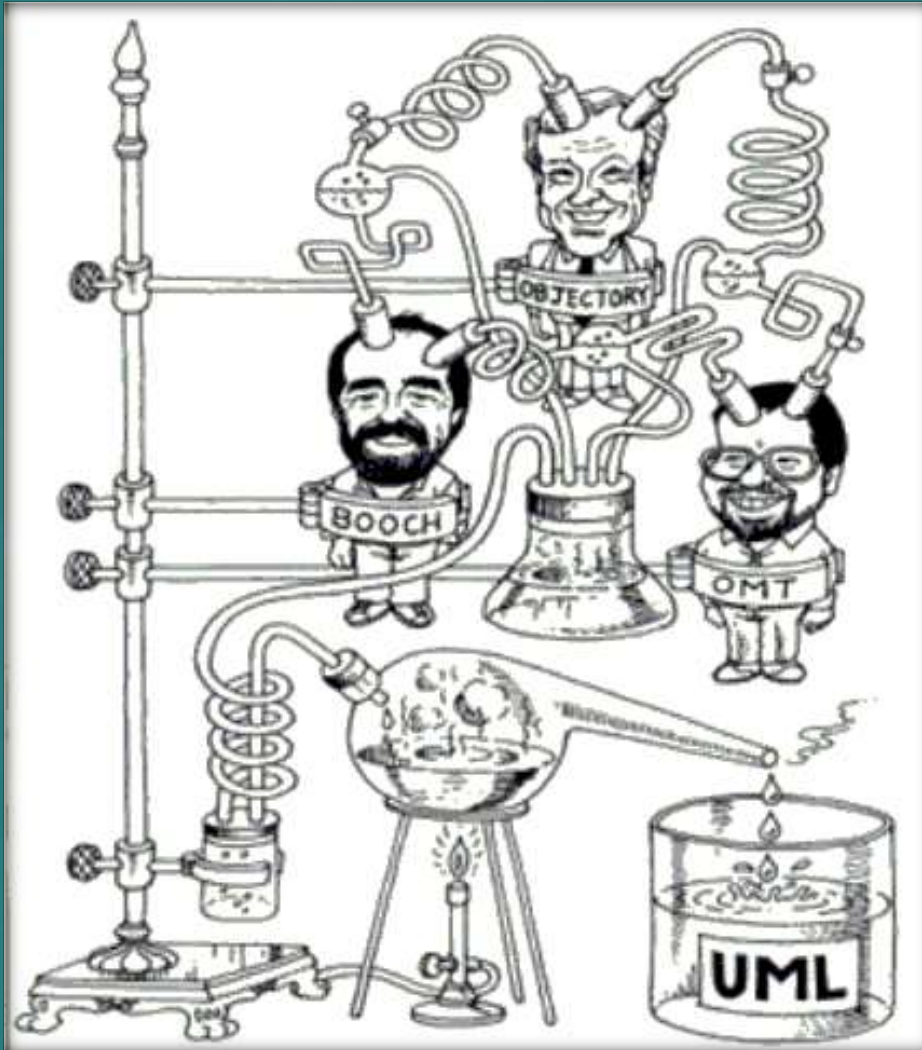




Small projects:
can generate a mental
modal on demand

models bigger roles in
... are indispensable for bigger projects

UML (Unified Modeling Language)



Grady Booch



Ivar Jacobson



James Rumbaugh

UML creators are also known as ...

- The gang of four
- The order of the white lotus
- The three amigos
- Spice boys



Grady Booch



Ivar Jacobson

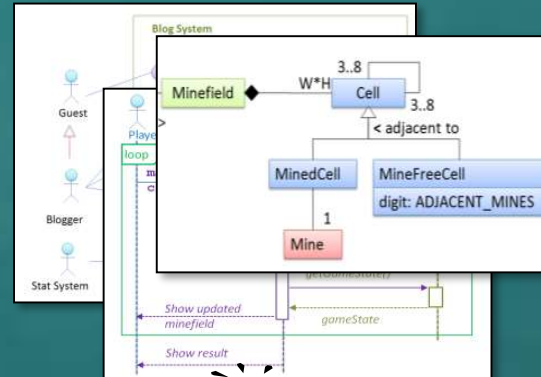
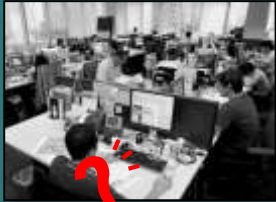


James Rumbaugh

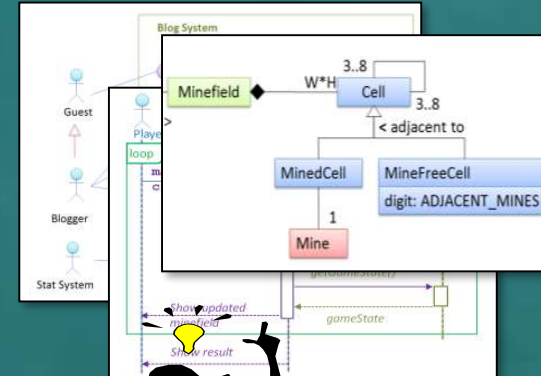


models for what?

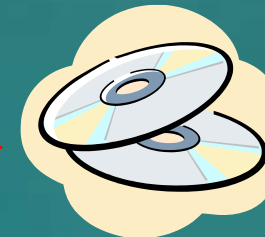
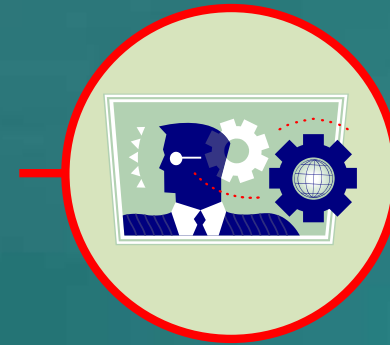
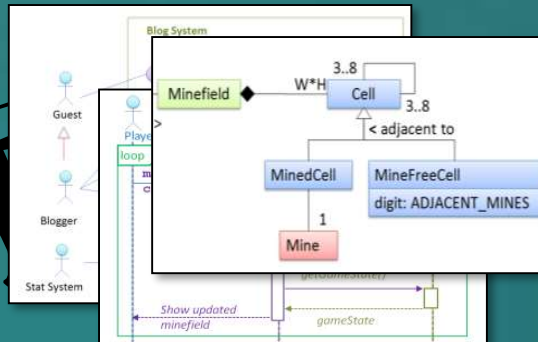
i. for analysis



ii. for communication



iii. as a blueprint



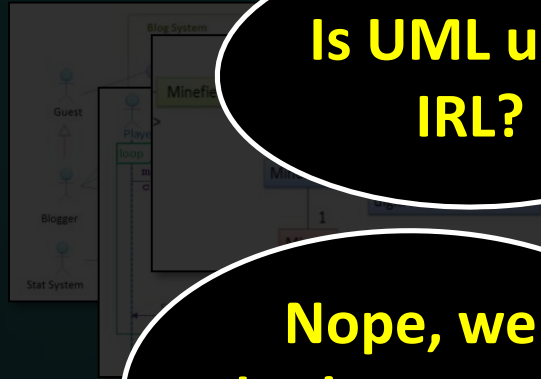
models for what?

i. for analysis

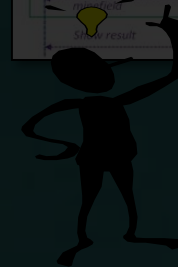
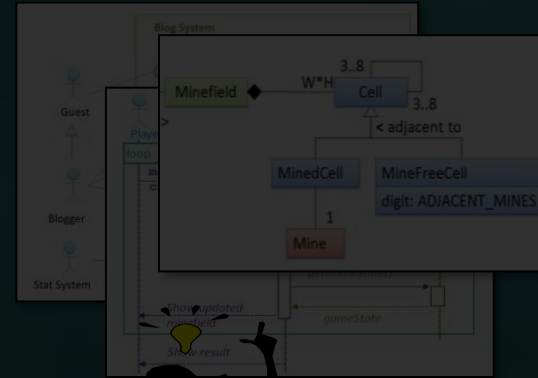


Is UML used
IRL?

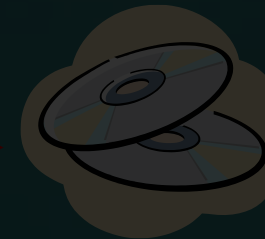
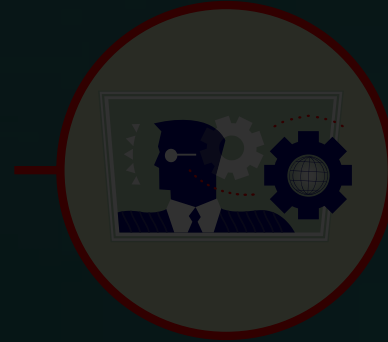
Nope, we
don't use UML
diagrams



ii. for communication



iii. as a blueprint



UML

[] There are better alternatives

? No modeling

? Ad-hoc notations

? Domain-specific models

e.g., a modelling notation specifically for finance domain

[] Not used a lot in actual practice

[] Over-emphasized in this course

The ability to visualize code in some other form (i.e. *model* code)
– **UML or other notations** – is an essential survival skill.

UML

Modeling **structure**

- Class structure:
class diagrams
- Object structure:
object diagrams

Modeling **behavior**

For the time being, *interpreting* only!
Diagram → implement code ? Not yet
Code → draw diagram? Not yet



Imagine this scenario
in project meeting at the workplace:

Ann: An engineer, college grad

- Goes to the whiteboard and ...

Betsy: An intern, college student

Derrick: Tech lead/ manager

Charlie: An engineer, self-taught



Imagine this scenario
in project meeting at the workplace:

Ann: An intern, college grad

- Goes to the whiteboard and ...

Betsy: An intern, college student

- “Yes, but that adds another layer of abstraction;
is it worth it?”

Derrick: Tech lead/ manager

- Nods in agreement

Charlie: An engineer, self-taught

- Silent

We aim to learn UML usage for ...

[] for sketching to aid discussion

[] for documenting

[] as blueprints

[] for code generation

[] for model checking

A formal methods technique that analyzes a model of a system to check whether given properties of a system are satisfied.

What prerequisite topics would you like CS2103 to cover?

CS4211 (Formal Methods for Software Engineering) Prof:

I assume they know UML; I often refer to UML diagrams like class/state diagrams.

CS3203 – (Software Engineering Project) Prof:

We expect students to model the project design using UML diagram before they start coding

Week 4

Topics:

- [W4.1] Design: Models ⚡⚡
- [W4.2] Class/Object Diagrams: Basics ⚡
- [W4.3] Class Diagrams: Intermediate-Level ⚡⚡
- [W4.4] Java: JavaFX ⚡⚡⚡
- [W4.5] Java: varargs 🎓
- [W4.6] Code Quality: Naming ⚡⚡
- [W4.7] Static Analysis ⚡⚡⚡
- [W4.8] Code reviews ⚡⚡⚡
- [W4.9] RCS: Managing Pull Requests I ⚡⚡

Admin:

Recommended to watch the videos

the module

iP:

1. Use GFMD in the PR description
2. Review some peer PRs [P](#)
3. Learn from others (optional)
4. Add Increments as branches: [A-CheckStyle](#), [Level-10](#), [A-Varargs](#)

tP:

1. 👤 Start weekly project meetings
2. 👤 Start a collaborative doc to take project notes
[🕒 before the tutorial](#)
3. 👤 Decide on an overall project direction (user profile, problem addressed) [🕒 before the tutorial](#)

Week 4

Topics:

- [W4.1] **Design: Models** ⚡⚡
- [W4.2] **Class/Object Diagrams: Basics** ⚡
- [W4.3] **Class Diagrams: Intermediate-Level** ⚡⚡
- [W4.4] **Java: JavaFX** ⚡⚡⚡
- [W4.5] **Java: varargs** 🎓
- [W4.6] **Code Quality: Naming** ⚡⚡
- [W4.7] **Static Analysis** ⚡⚡⚡
- [W4.8] **Code reviews** ⚡⚡⚡
- [W4.9] **RCS: Managing Pull Requests I** ⚡⚡

Admin:

1. Accept GitHub invitation from the module organization [P](#)
2. Submit post-lecture quiz [P](#)

iP:

1. Use GFMD in the PR description
2. Review some peer PRs [P](#)
3. Learn from others (optional)
4. Add Increments as branches: [A-CheckStyle](#), [Level-10](#), [A-Varargs](#)

tP:

1. 👤 Start weekly project meetings
2. 👤 Start a collaborative doc to take project notes
[🕒 before the tutorial](#)
3. 👤 Decide on an overall project direction (user profile, problem addressed) [🕒 before the tutorial](#)

Week 4

Topics:

- [W4.1] **Design: Models** ⚡⚡
- [W4.2] **Class/Object Diagrams: Basics** ⚡
- [W4.3] **Class Diagrams: Intermediate-Level** ⚡⚡
- [W4.4] **Java: JavaFX** ⚡⚡⚡
- [W4.5] **Java: varargs** 🎓
- [W4.6] **Code Quality: Naming** ⚡⚡
- [W4.7] **Static Analysis** ⚡⚡⚡
- [W4.8] **Code reviews** ⚡⚡⚡
- [W4.9] **RCS: Managing Pull Requests I** ⚡⚡

Admin:

1. Accept GitHub invitation from the module organization **P**
2. Submit post-lecture quiz **P**

iP:

1. Use GFMD in the PR description
2. Review some peer PRs **P**
3. Learn from others (optional)
4. Add Increments as branches: **A-CheckStyle**, **Level-10**, **A-Varargs**

tP:

1. 👤 Start weekly project meetings
2. 👤 Start a collaborative doc to take project notes
🕒 before the tutorial
3. 👤 Decide on an overall project direction (user profile, problem addressed) **🕒 before the tutorial**

Week 4

Topics:

- [W4.1] **Design: Models** ⚡⚡
- [W4.2] **Class/Object Diagrams: Basics** ⚡
- [W4.3] **Class Diagrams: Intermediate-Level** ⚡⚡
- [W4.4] **Java: JavaFX** ⚡⚡⚡
- [W4.5] **Java: varargs** 🎓
- [W4.6] **Code Quality: Naming** ⚡⚡
- [W4.7] **Static Analysis** ⚡⚡⚡
- [W4.8] **Code reviews** ⚡⚡⚡

Admin:

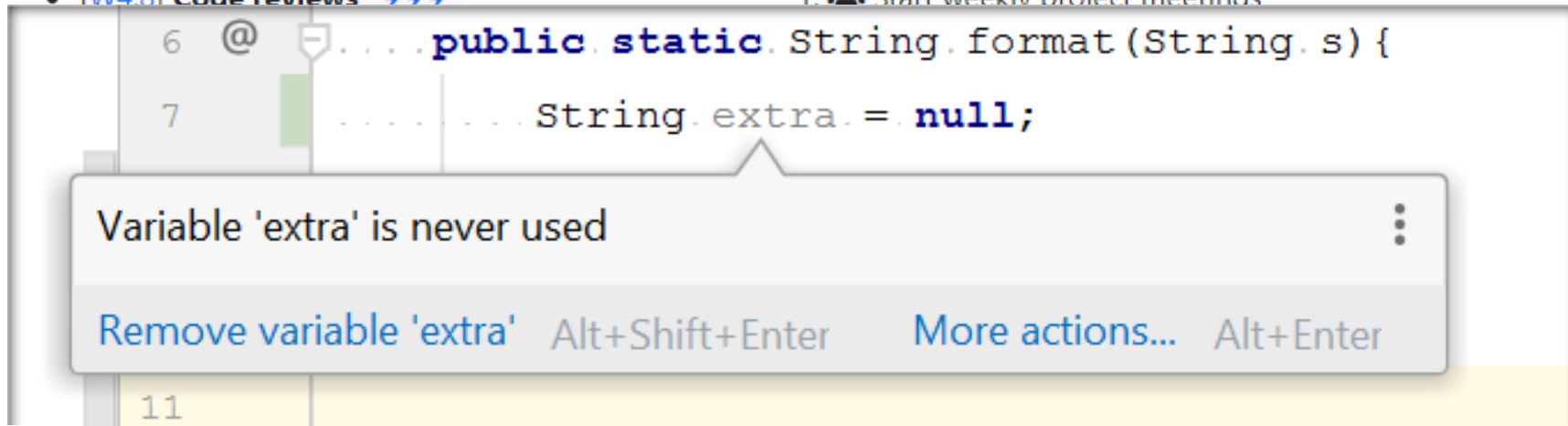
1. Accept GitHub invitation from the module organization [P](#)
2. Submit post-lecture quiz [P](#)

iP:

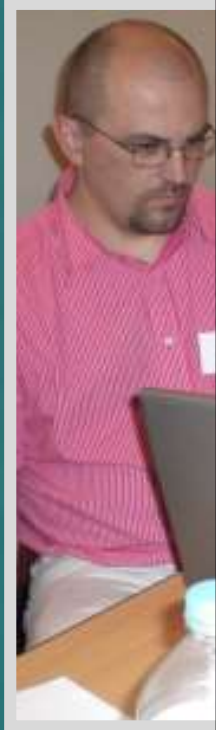
1. Use GFMD in the PR description
2. Review some peer PRs [P](#)
3. Learn from others (optional)
4. Add Increments as branches: [A-CheckStyle](#), [Level-10](#), [A-Varargs](#)

tP:

1. 👤 Start weekly project meetings



Code reviews



Code reviews



Q₇

It's harder to learn the _____ of code reviews

() mechanics

() tools

() inter-personal aspects

give constructive feedback,
without offending

Receive feedback,
without getting offended

Respond to feedback,
without getting defensive

Week 4

Topics:

- [W4.1] **Design: Models** ⚡⚡
- [W4.2] **Class/Object Diagrams: Basics** ⚡
- [W4.3] **Class Diagrams: Intermediate-Level** ⚡⚡
- [W4.4] **Java: JavaFX** ⚡⚡⚡
- [W4.5] **Java: varargs** 🎓
- [W4.6] **Code Quality: Naming** ⚡⚡
- [W4.7] **Static Analysis** ⚡⚡⚡
- [W4.8] **Code reviews** ⚡⚡⚡
- [W4.9] **RCS: Managing Pull Requests I** ⚡⚡

Admin:

1. Accept GitHub invitation from the module organization **P**
2. Submit post-lecture quiz **P**

iP:

1. Use GFMD in the PR description
2. Review some peer PRs **P**
3. Learn from others (optional)
4. Add Increments as branches: **A-CheckStyle**, **Level-10**, **A-Varargs**

tP:

1. 👤 Start weekly project meetings
2. 👤 Start a collaborative doc to take project notes
🕒 before the tutorial
3. 👤 Decide on an overall project direction (user profile, problem addressed) **🕒 before the tutorial**

Week 4

Topics:

- [W4.1] **Design: Models** ⚡⚡
- [W4.2] **Class/Object Diagrams: Basics** ⚡
- [W4.3] **Class Diagrams: Intermediate-Level** ⚡⚡
- [W4.4] **Java: JavaFX** ⚡⚡⚡
- [W4.5] **Java: varargs** 🎓
- [W4.6] **Code Quality: Naming** ⚡⚡
- [W4.7] **Static Analysis** ⚡⚡⚡
- [W4.8] **Code reviews** ⚡⚡⚡
- [W4.9] **RCS: Managing Pull Requests I** ⚡⚡

Admin:

1. Accept GitHub invitation from the module organization **P**
2. Submit post-lecture quiz **P**

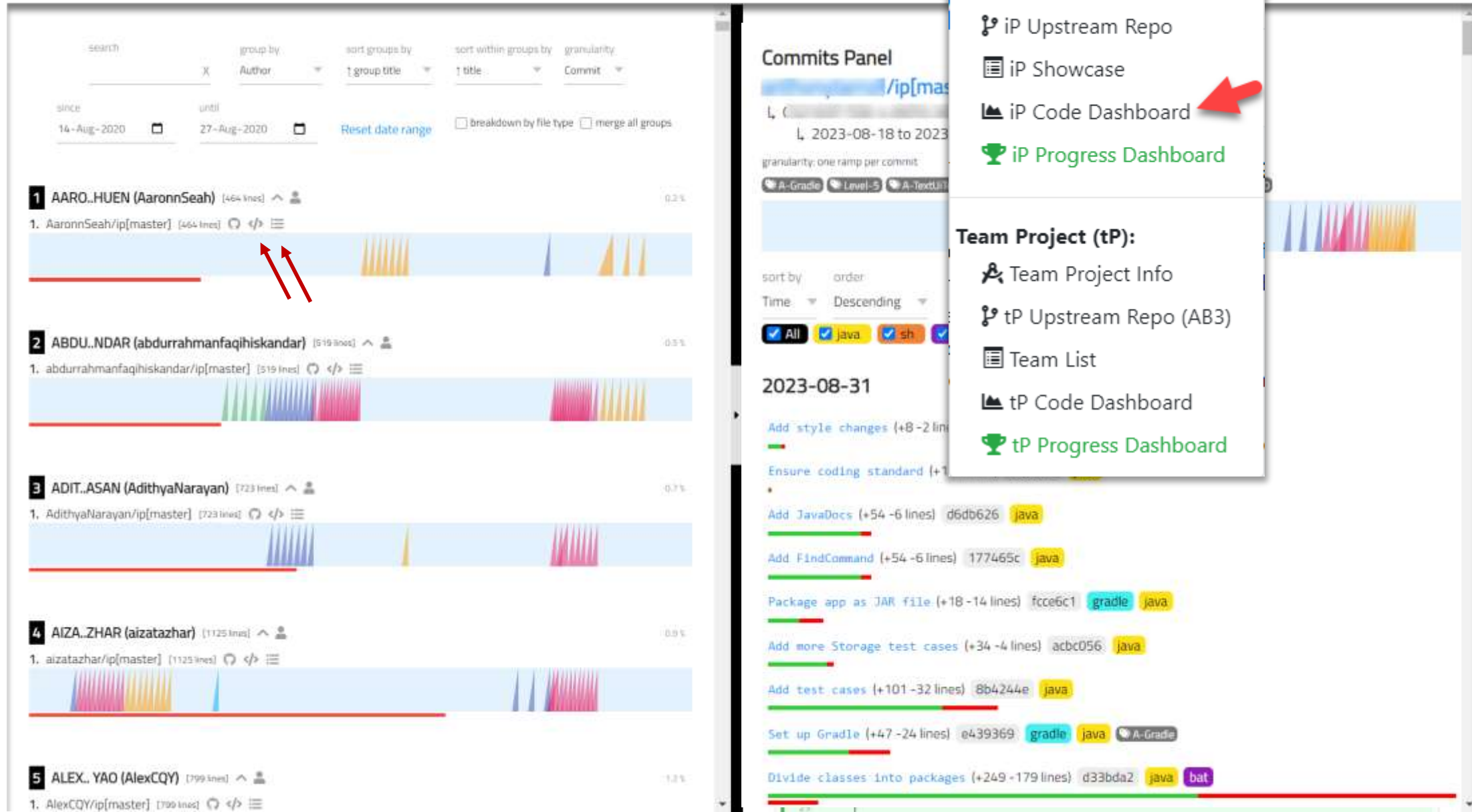
iP:

1. Use GFMD in the PR description
2. Review some peer PRs **P**
3. Learn from others (optional)
4. Add Increments as branches: **A-CheckStyle**, **Level-10**, **A-Varargs**

tP:

1. 👥 Start weekly project meetings
2. 👥 Start a collaborative doc to take project notes
🕒 before the tutorial
3. 👥 Decide on an overall project direction (user profile, problem addressed) **🕒 before the tutorial**

RepoSense code dashboard for the iP



Project Links ▾ Other Links ▾

Individual Project (iP):

- Individual Project Info
- iP Upstream Repo
- iP Showcase
- iP Code Dashboard
- iP Progress Dashboard

Team Project (tP):

- Team Project Info
- tP Upstream Repo (AB3)
- Team List
- tP Code Dashboard
- tP Progress Dashboard

Recommended: ____ look at others' code

☐ Don't

☐ Please

In the area of AI-assisted coding, reviewing not-your-own code (e.g., code generated by AI) is an even more valuable skill.

[but follow our reuse policies]

[automated plagiarism checks on **tP** (not iP) code]

Week 4

Topics:

- [W4.1] **Design: Models** ⚡⚡
- [W4.2] **Class/Object Diagrams: Basics** ⚡
- [W4.3] **Class Diagrams: Intermediate-Level** ⚡⚡
- [W4.4] **Java: JavaFX** ⚡⚡⚡
- [W4.5] **Java: varargs** 🎓
- [W4.6] **Code Quality: Naming** ⚡⚡
- [W4.7] **Static Analysis** ⚡⚡⚡
- [W4.8] **Code reviews** ⚡⚡⚡
- [W4.9] **RCS: Managing Pull Requests I** ⚡⚡

Watch the 'tP – part 1' course briefing video segment (given in W4 schedule page)

Admin:

1. Accept GitHub invitation from the module organization **P**
2. Submit post-lecture quiz **P**

iP:

1. Use GFMD in the PR description
2. Review some peer PRs **P**
3. Learn from others (optional)
4. Add Increments as branches: **A-CheckStyle**, **Level-10**, **A-Varargs**

tP:

1. 👤 Start weekly project meetings
2. 👤 Start a collaborative doc to take project notes
🕒 before the tutorial
3. 👤 Decide on an overall project direction (user profile, problem addressed) **🕒 before the tutorial**



*Thank
you!*

Dr Ganesh Neelakanta Iyer

gni@nus.edu.sg

<http://ganeshniyer.github.io>

<https://www.linkedin.com/in/ganeshniyer/>